

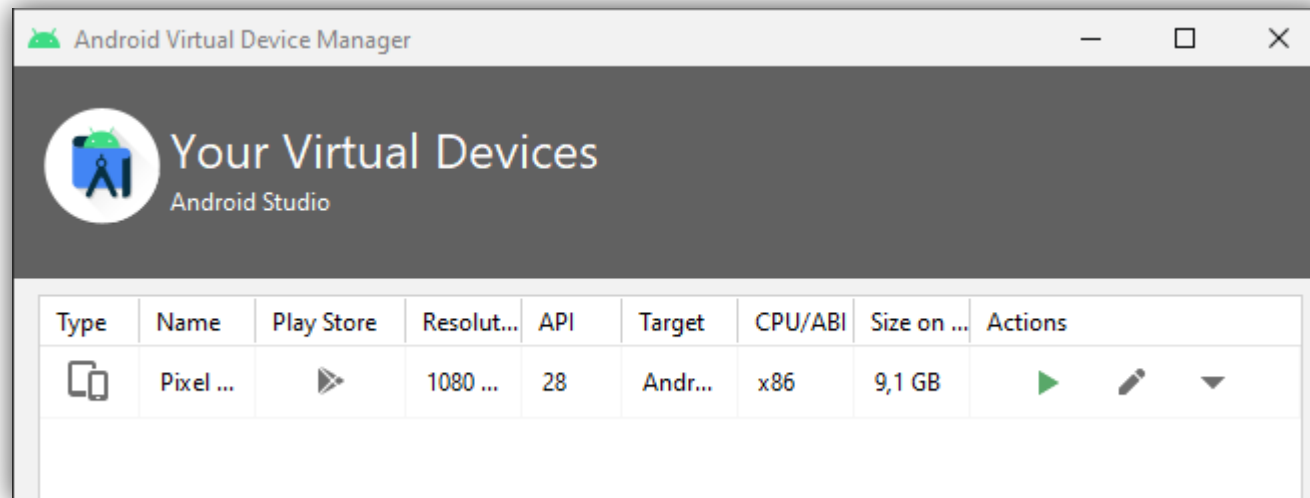
PROGRAMAÇÃO DE DISPOSITIVOS MÓVEIS

Prof. Tiago Piperno Bonetti
bonetti@prof.unipar.br

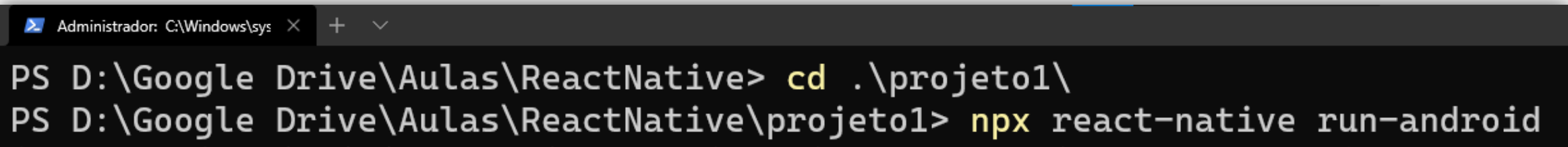


Criando e executando o projeto

```
npm install react-native@latest x + v
PS D:\> cd '.\Google Drive\Aulas\ReactNative\'
PS D:\Google Drive\Aulas\ReactNative> npx react-native init projeto1
```

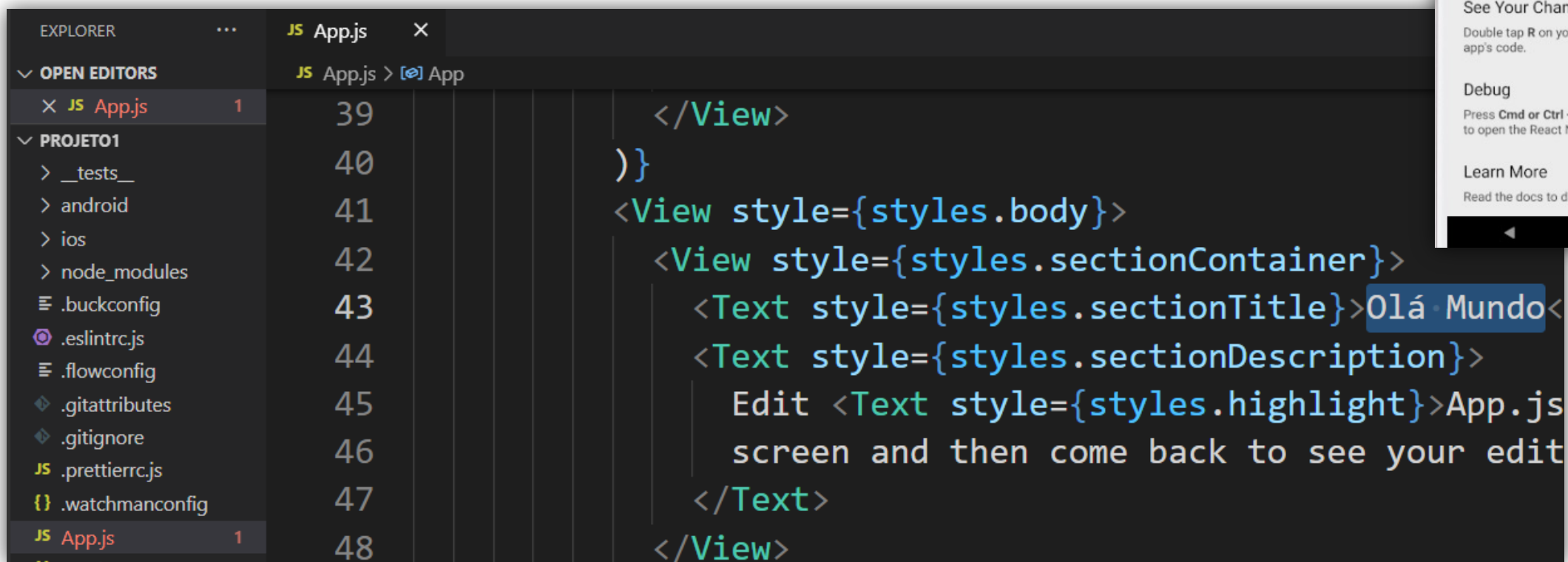


Criando e executando o projeto

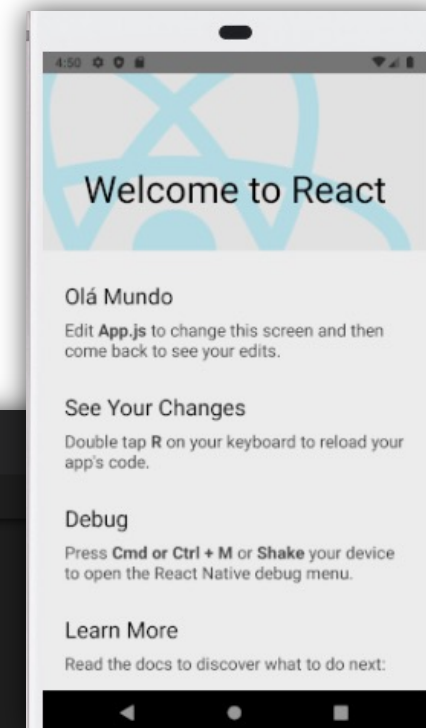


```
Administrador: C:\Windows\sys x + v
PS D:\Google Drive\Aulas\ReactNative> cd .\projeto1\
PS D:\Google Drive\Aulas\ReactNative\projeto1> npx react-native run-android
```

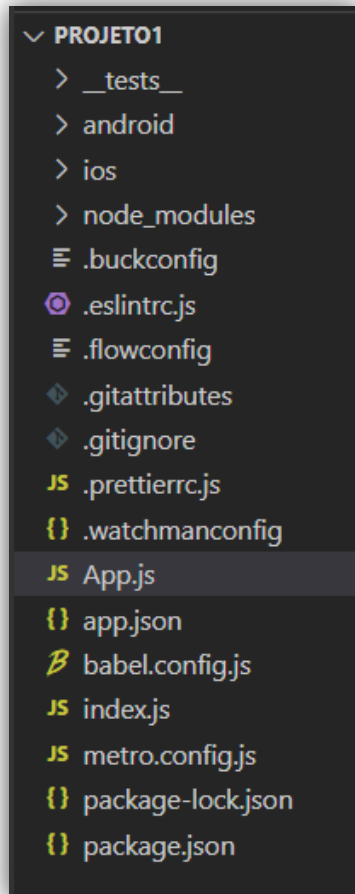
Abrindo no VS



```
39 </View>
40 )}
41 <View style={styles.body}>
42   <View style={styles.sectionContainer}>
43     <Text style={styles.sectionTitle}>Olá Mundo<
44     <Text style={styles.sectionDescription}>
45       Edit <Text style={styles.highlight}>App.js
46       screen and then come back to see your edit
47     </Text>
48   </View>
```



Estrutura



/node_modules: estão contidas as dependências do projeto.

App.js: é o arquivo (e componente) ponto de entrada do projeto.

Aqui vale lembrar que o React também é caracterizado por ser uma tecnologia de Single Page Application (SPA), ou seja, na prática sempre o que teremos no aplicativo (ou na solução web) é uma única página onde os componentes são renderizados dando a impressão de que o usuário está navegando por páginas diferentes.

app.json: contém metadados do aplicativo, tais como: nome, versão, dependências, entre outras configurações do projeto.

Index.js

```
JS App.js JS index.js ×
JS index.js > ...
1  /**
2   * @format
3   */
4
5  import {AppRegistry} from 'react-native';
6  import App from './App';
7  import {name as appName} from './app.json';
8
9  AppRegistry.registerComponent(appName, () => App);
10
```

5: importa do react-native

6: importa o app.js

7: importa o json

9: registra o componente raiz da aplicação

Componentes funcionais

```
JS App.js > ...
1  import React from 'react';
2  import { Text } from 'react-native';
3  //Importa um componente do react-native.
4
5  function App() { //cria uma função
6
7      return <Text>Componente 1</Text>
8      //Retorna um componente usando jsx
9  }
10
11  export default App;
12  //exporta a função para index.js
```

O componente **começa** como uma **função**.

Tudo o que um componente de função **retorna** é **processado** como um **elemento** React.

Aqui, o componente App renderizará um **elemento** **<Text>**.

Você pode exportar seu componente de função com JavaScript **export default** para uso em todo o seu aplicativo.

Componentes funcionais

```
5  const App = function () {  
6    //cria uma função anônima  
7    //associamos a função a constante App  
8  
9    return <Text>Componente 1</Text>  
10   //Retorna um componente usando jsx  
11 }
```

```
5  const App = () => {  
6    //cria uma função anônima  
7    //associamos a função a constante App  
8  
9    return <Text>Componente 1</Text>  
10   //Retorna um componente usando jsx  
11 }
```



```
JS Primeiro.js X JS App.js
components > JS Primeiro.js > ...
1 import React from 'react'
2 import { Text } from 'react-native'
3
4 const CompPrimeiro = () => {
5
6     return <Text>Esse é o nosso
7         primeiro componente</Text>
8
9 }
10
11 export default CompPrimeiro
```

```
Primeiro.js JS App.js X
App.js > ...
1 import React from 'react'
2 import { Text } from 'react-native'
3
4 import CompPrimeiro from './components/Primeiro'
5 //importando o componente que criamos
6
7 const App = () => {
8     return <CompPrimeiro/>
9
10 }
11
12 export default App
```

Componentes

```
1  import React from 'react';
2  import { Text } from 'react-native';
3
4  const Hello = () => {
5    const nome = 'Tiago';
6    //cria uma constante para armazenar o nome
7
8    return <Text>Olá, eu sou o {nome}!</Text>;
9    //retorna usando a constante
10 };
11
12 export default Hello;
```

Como **JSX** é JavaScript, você pode usar **variáveis** dentro dele.

Aqui você está declarando um nome em “nome”, e incorporando-o com chaves dentro <Text>.

Componentes

Adicionando o novo componente ao App.

```
1  import React from 'react';
2  import { Text, View } from 'react-native';
3  //adicionamos o comp View
4
5  import CompPrimeiro from './components/primeiro'
6  import Hello from './components/hello'
7  // importamos o novo componente
8
9  const App = () => {
10
11    return (
12      <View>
13        { /* criamos uma view */ }
14        <CompPrimeiro/>
15        <Hello/>
16        { /* adicionamos o novo componente */ }
17      </View>
18    )
19  }
20
21  export default App;
```

Componentes

```
1  import React from 'react';
2  import { Text } from 'react-native';
3
4  const getNomeCompleto = (primeiroNome, segundoNome) => {
5    // criamos uma função
6    return primeiroNome + " " + segundoNome ;
7    // a função retorna o nome completo
8  }
9
10 const Hello = () => {
11
12   return (
13     <Text>
14       Olá, eu sou o {getNomeCompleto("Tiago", "Bonetti")}!
15     </Text>
16     // estamos mostrando o nome usando a função
17   );
18 };
19
20 export default Hello;
```

Qualquer **expressão JavaScript** funcionará **entre chaves**, incluindo chamadas de função como:

`{getNomeCompleto("Tiago", "Bonetti")}`.

Componentes

```
1  import React from 'react';
2  import { Text, TextInput, View } from 'react-native';
3
4  const Hello = () => {
5    return (
6      <View>
7        <Text>Olá, eu sou...</Text>
8        <TextInput
9          style={{
10            height: 40,
11            borderColor: 'gray',
12            borderWidth: 1,
13          }}
14          defaultValue="Nomeie-me!"
15        />
16      </View>
17    );
18  };
19
20  export default Hello;
```

O React permite **aninhar** esses **componentes uns dentro dos outros** para criar novos componentes.

Esses componentes **aninhados** e **reutilizáveis** estão no **cerne** do **paradigma React**.

Por exemplo, você pode aninhar Text e TextInput dentro de um View, e o React Native irá renderizá-los juntos.

Múltiplos Componentes

Você pode **renderizar** esse componente **várias vezes** e **vários lugares** sem repetir seu código.

Qualquer componente que renderiza outros componentes é um **componente pai**.

Aqui, App está o componente pai e cada Hello é um componente filho.

Cada <Hello> renderiza um elemento único - que você pode personalizar com props.

```
1  import React from 'react';
2  import { Text, View } from 'react-native';
3
4  const Hello = () => {
5    return (
6      <View>
7        <Text>Eu sou o Tiago.</Text>
8      </View>
9    );
10 };
11
12 const App = () => {
13   return (
14     <View>
15       <Text> Seja Bem vindo!</Text>
16       <Hello />
17       <Hello />
18       <Hello />
19     </View>
20   );
21 };
22
23 export default App;
```

Props

Props é a abreviação de "**propriedades**".

Props permitem que você **personalize** os **componentes** do React.

Por exemplo, aqui você passa cada <Hello> um diferente nome para <Hello> renderizar:

```
1  import React from 'react';
2  import { Text, View } from 'react-native';
3
4  const Hello = (props) => {
5    return (
6      <View>
7        <Text>Eu sou o {props.nome}</Text>
8      </View>
9    );
10 };
11
12 const App = () => {
13   return (
14     <View>
15       <Text> Seja Bem vindo!</Text>
16       <Hello nome="Tiago" />
17       <Hello nome="João" />
18       <Hello nome="Murilo" />
19     </View>
20   );
21 };
22
23 export default App;
```

Estado

Enquanto você possa pensar em **props** como **argumentos** usados para **configurar** como os componentes são renderizados, o **estado** é como o armazenamento de **dados pessoais** de um **componente**.

O estado é útil para lidar com **dados** que **mudam** com o **tempo** ou que vêm da **interação** do usuário.

Estado dá **memória** aos seus **componentes**!

Como **regra geral**, use **props** para **configurar** um componente **durante** a **renderização**. Use o **estado** para **controlar** todos os **dados** do componente que você espera que **mudem** com o tempo.

Estado

```
1  import React, { useState } from 'react';
2  // importamos o useState
3  import { Button, Text, View } from 'react-native';
4
5  const Cliente = (props) => {
6    const [sede, setSede] = useState(true);
7    // Declara o estado do componente chamando useState.
8    // Cria uma "variável de estado" com um valor inicial.
9    // A variável de estado é "sede" seu valor inicial é true.
10   // Cria uma função para definir o valor dessa variável de estado "setSede".
```

Você pode adicionar estado a um componente chamando o **useState** Hook do React .

Um Hook é um tipo de função que permite que você “**se conecte**” aos recursos do React.

Por exemplo, **useState** é um Hook que permite adicionar **estado** aos **componentes** da função.

```

5  const Cliente = (props) => {
6    const [sede, setSede] = useState(true);
7    // Declara o estado do componente chamando useState.
8    // Cria uma "variável de estado" com um valor inicial.
9    // A variável de estado é "sede" seu valor inicial é true.
10   // Cria uma função para definir o valor dessa variável de estado "setSede".
11
12   return (
13     <View>
14       <Text>
15         Eu sou {props.name}, e estou {sede ? 'com sede' : 'bem, obrigado'}!
16       </Text>
17
18       <Button
19         onPress={() => {
20           setSede(false);
21         }}
22         disabled={!sede}
23         title={sede ? 'Sirva-me um pouco de água, por favor! ' : ' Obrigado!'}
24       />
25     </View>
26   );
27 };

```

```

29  const App = () => {
30    return (
31      <>
32        <Cliente name="Tiago" />
33        <Cliente name="Murilo" />
34      </>
35    );
36  };
37
38  export default App;

```

Referências

Escudelario, Bruna de Freitas; Pinho, Diego Martins. **React Native: Desenvolvimento de aplicativos mobile com React**. Casa do código.

Documentação React Native. Disponível em: <https://reactnative.dev/docs>